

Session 6 — Study Guide: Express Structure & Request Handling

This guide explains in depth what changed in B4F Hub in Session 6 and, more importantly, **why**. If you forgot most of the class, you should be able to read this alone and rebuild your understanding. The short version is in the Student Notes; this is the long version.

Before this session began, Session 5's continuity homework (`GET /api/health` , `GET /api/opportunities/type/:type` , the Navbar health summary) was reviewed and its official solution taught live, so everyone started today with the same fully-working backend Session 5 left.

1. The real problem: a file that outgrew itself

`server/index.js` at the start of this session was well over 200 lines, mixing three unrelated domains in one file: Health, Community (posts), and Opportunities. It still worked exactly as well as it always had — nothing was broken. What had changed was readability: fixing a bug in "how posts are liked" meant scrolling past health checks and opportunity routes that have nothing to do with it.

This is the actual, honest motivation for everything in this session. **Express does not require** any particular file structure — a single-file server is completely valid Express. Today's work is purely for the humans reading the code, not a rule the framework enforces.

2. `express.Router()` is not a second server

`express.Router()` returns a small object that can register routes exactly the way `app` does (`router.get(path, handler)` , `router.patch(path, handler)`) — but on its own it does nothing. It has to be **mounted** onto the real app with `app.use(mountPath, router)` before any request can reach it. There is still exactly one `app` , one `app.listen(...)` , one port, one process.

The key rule: a route declared inside a router is **relative to the mount path** it's given.

TEXT

```
app.use("/api/opportunities", opportunitiesRouter);
                                |
router.get("/type/:type", handler) --> full path: /api/opportunities/type/:type
```

Mount path + relative path = full path. The mount path is supplied exactly once, by `app.use(...)` in `index.js` ; it is never repeated inside the router's own route declarations.

3. `req.body` , in depth, from the real `POST /api/posts`

Nothing about `req.body` is new this session — it already existed. What's new is looking at it closely, using the already-working handler (at the time of this discussion it is still written inline in `server/index.js` as `app.post("/api/posts", ...)` ; by the end of the session, this same logic — unchanged — lives in `server/controllers/posts.js` as `createPost`):

```
export function createPost(req, res) {
  const { content, category } = req.body ?? {};

  if (typeof content !== "string") {
    return res
      .status(400)
      .json({ error: "content is required and must be a string." });
  }
  // ...trim, length checks, category check, each an early return...
}
```

`?? {}` matters: if a request arrives with no body at all, `req.body` can be `undefined`, and destructuring `content / category` directly off `undefined` would throw before any validation runs — crashing the request instead of answering it cleanly with a `400`.

Each check `return`s immediately on failure — an early return, so nothing later in the function, and never the real `posts` array, is touched once something is already wrong.

The one idea worth remembering permanently: React's own form already stops a user from submitting an empty post before this backend code ever runs. So why does the backend check again? Because `POST /api/posts` is a URL, not a page. curl, Postman, a browser's dev console, or a completely different application can call it directly, with none of the frontend's checks in the way. **Frontend validation is for UX. Backend validation protects the API boundary** — it is the only check that cannot be bypassed by skipping the frontend entirely.

4. Status codes, from real examples already in this project

Cod e	What it means	Real example
200	Success, nothing went wrong	<code>GET /api/opportunities</code>
201	A new resource was created	<code>POST /api/posts</code> 's last line
400	The client sent something invalid	any validation check
404	Nothing exists at that path/id	<code>GET /api/opportunities/:id</code> with no match
409	The resource exists, but this action conflicts with its current state	applying twice to the same opportunity

400 vs. 404: "what you sent is invalid" vs. "what you sent was fine, but nothing matches it." 404 vs. 409: "nothing there at all" vs. "something's there, but this specific action can't happen right now." This is not a memorization exercise — every code here already had a real, correct use before today; the only job was naming what was already there.

5. Who owns mutable runtime state?

Once route handlers move into their own files, they still need to read (and sometimes change) the *same* data `index.js` already used. `data.js` defines the seed data the app starts with; `store.js` exists to be the one explicit, intentional owner of the *current, mutable runtime state* built from it — not because two files importing the same thing from `data.js` would somehow diverge on their own (they wouldn't — see below), but so that state has one clear home instead of being scattered across whichever files happen to reach for it.

```
JS
// server/store.js (shown here as CORE first left it — the teamwork adds posts alongside it)
import { initialPosts, initialOpportunities } from "../data.js";

export const opportunities = initialOpportunities;
export const ALLOWED_TYPES = ["job", "internship", "scholarship", "volunteer"];
```

Every file that needs `opportunities` imports it from `store.js`, never `initialOpportunities` directly from `data.js` — by convention, for a clear architectural boundary, not because a direct import would actually behave differently today (ES modules evaluate a module once, so every importer of the same export shares the identical object). The real risks this boundary protects against are a *future* independent copy built somewhere with `.map((o) => ({ ...o }))`, or a local reassignment that only one file's own binding would see — either would silently create two arrays where the architecture assumes one. Because `opportunities` itself is never *reassigned* — only individual objects inside it are mutated (`opportunity.applied = true`) — a plain `export const` is enough here.

`posts` is different: the original code reassigned it (`posts = [newPost, ...posts]`). A module can only reassign a binding it declares itself, never one it imported — so the teamwork's own extraction changed this one line to `posts.unshift(newPost)` (an in-place mutation) instead, and added a small `claimNextPostId()` function in `store.js` to own the id counter safely. Same underlying rule, one small extra step because `posts` genuinely needed it and `opportunities` didn't.

A small utility function (`delay()`) gets the exact same treatment, in `server/utils.js` — not everything worth sharing between files is data.

6. A controller is just a plain file of plain functions

```
JS
// server/controllers/opportunities.js
import { opportunities, ALLOWED_TYPES } from "../store.js";
import { delay } from "../utils.js";

export async function getAllOpportunities(req, res) {
  await delay(350);
  res.json(opportunities);
}
```

Nothing about this filename or shape is special to Express — it's a plain JavaScript module exporting plain `(req, res)` functions, moved verbatim from `index.js`. Express never sees this file directly; only wiring it into a route (`next`) makes it reachable.

7. Wiring it together: the Router file

JS

```
// server/routes/opportunities.js
import express from "express";
import {
  getAllOpportunities,
  getOpportunitiesByType,
  getOpportunityById,
  applyToOpportunity,
} from "../controllers/opportunities.js";

const router = express.Router();

router.get("/", getAllOpportunities);
router.get("/type/:type", getOpportunitiesByType);
router.get("/:id", getOpportunityById);
router.patch("/:id", applyToOpportunity);

export default router;
```

Every path here is shorter than the original `app.get("/api/opportunities/...")` form, because the `/api/opportunities` prefix is supplied once, by `app.use(...)` in `index.js`. `/type/:type` is still registered before `/:id` — Express matches routes in registration order, so a general pattern like `:id` registered first would swallow `type` as if it were an id value.

8. The payoff, in `index.js`

JS

```
// ----- Opportunities -----
app.use("/api/opportunities", opportunitiesRouter);
```

One line replaces roughly sixty. By the end of the session (after the teamwork and the homework), `index.js` is a thin composition root: imports, two lines of middleware, and router mounts — nothing else. Every request behaves identically to before the refactor; this was proven at every step with the exact same curl commands, before and after.

9. FLEX: a fourth kind of middleware (discussed, not built)

Express recognizes a middleware with exactly **four** parameters, `(err, req, res, next)`, as special: an error handler, called when something throws or calls `next(err)`, instead of running in the normal flow. This app doesn't have one — and doesn't need one yet, since every real failure case this session covers

(400/404/409) is already handled deliberately, by name, inside the route itself. An error-handling middleware is a later tool, for genuinely unexpected crashes, not a replacement for deliberate status-code decisions.

Glossary

- **Router** — an `express.Router()` object grouping related routes, mounted onto the real app.
- **Mount path** — the path prefix given to `app.use(mountPath, router)`; every route inside that router is relative to it.
- **Controller** — a plain file exporting plain `(req, res)` functions; no special Express meaning.
- **Single source of truth** — one real, shared array/value every file reads from, never a per-file copy.
- **Early return** — `return` ing immediately once a validation check fails, so nothing after it runs.
- **Status code** — a number in the HTTP response naming the outcome (200/201/400/404/409 this session).

Review questions

1. If `router.get("/:id", handler)` is mounted with `app.use("/api/posts", router)`, what is the real, full path a client must request?
2. Why does `posts` need a `claimNextPostId()` function in `store.js`, while `opportunities` never needed anything like it?
3. What's the difference between a 400 and a 404? Between a 404 and a 409?
4. Why does the backend validate `POST /api/posts`'s body even though React's own form already checks it?
5. What would go wrong if `controllers/opportunities.js` imported `initial0pportunities` directly from `data.js` instead of `opportunities` from `store.js`?

What you should be able to explain now

- Why a server file that worked fine can still be worth restructuring.
- What a Router actually is, and why it is not a second server.
- The full extraction sequence: controller function, router wiring, `app.use` mount, remove the old inline route.
- Why shared, mutable state needs one real owner, and what changes when that state is reassigned instead of mutated in place.
- Which of the five status codes used this session applies to a given real situation, and why.

Not yet

- A database of any kind (MongoDB, PostgreSQL, Mongoose, Prisma).
- NestJS, Next.js, authentication, CORS configuration, `req.query`.
- A real, implemented Express error-handling middleware (discussed conceptually only).
- A validation library (Zod, Joi, express-validator) — every check this session is plain `if / typeof`.